

# Agente Conversacional Corporativo — Colgate-Palmolive Colombia

Agente de IA desplegado en WhatsApp que atiende consultas de consumidores las 24 horas mediante RAG sobre datos reales de la empresa, con vigilancia competitiva autónoma ejecutándose en segundo plano.

Python3.14

OpenFang0.6.9

Mistralsmall-latest

LicenciaAcadémica

| Campo       | Detalle                                                                                                   |
|-------------|-----------------------------------------------------------------------------------------------------------|
| Institución | Universidad Autónoma de Occidente                                                                         |
| Asignatura  | Técnicas de Inteligencia Artificial                                                                       |
| Módulo      | 3 · Ruta B: Sistema Operativo Agéntico                                                                    |
| Autores     | Jorge Castaño López                                                                                       |
|             | Natalia Arias Londoño                                                                                     |
|             | Jorge Mario Gallego Uribe                                                                                 |
|             | Jhonathan Leandro Clavijo Troches                                                                         |
| Fecha       | 5 de junio de 2026                                                                                        |
| Repositorio | <a href="https://github.com/jorgegallegou/colgatefinal">https://github.com/jorgegallegou/colgatefinal</a> |

## Descripción general

Este proyecto implementa y documenta el diseño, implementación y validación en producción de un agente conversacional corporativo para Colgate-Palmolive Colombia, desarrollado como entregable del Módulo 3 del curso de Técnicas de Inteligencia Artificial.

El proyecto se ejecuta bajo la **Ruta B: Sistema Operativo Agéntico**, desplegando el agente sobre OpenFang 0.6.9 — un kernel especializado para agentes de IA escrito en Rust. El sistema resultante es un bot de WhatsApp disponible las 24 horas que responde consultas de consumidores con conocimiento real de la empresa y ejecuta vigilancia competitiva autónoma en segundo plano.

Capacidades principales:

- **Respuestas en tiempo real** — productos, puntos de venta, historia, sostenibilidad y contacto
- **Aislamiento de sesiones por usuario** — cada número de teléfono tiene su propio contexto de conversación
- **Recuperación semántica (RAG)** — 235 fragmentos indexados en un Vector Store con Mistral Embeddings
- **Hands autónomos** — agentes en background que recolectan inteligencia competitiva cada 6 horas

## Contexto y Justificación

Colgate-Palmolive Colombia atiende consultas a través de su línea gratuita **018000520800**, operativa de lunes a viernes entre las 8:00 AM y las 6:00 PM. Fuera de ese horario no existe ningún canal digital automatizado disponible para el consumidor.

Las consultas más frecuentes que el agente resuelve son:

- Localización de puntos de venta por ciudad
- Composición e información nutricional de productos
- Historia, valores y estructura corporativa
- Datos de contacto, sedes y horarios de atención

El agente ofrece cobertura continua sin intervención humana, con respuestas fundamentadas en información oficial verificada.

## Selección de Ruta: A vs B

El Módulo 3 presentaba dos rutas de implementación:

| Dimensión                 | Ruta A — Agente Custom                                | Ruta B — Agent OS (OpenFang)                         |
|---------------------------|-------------------------------------------------------|------------------------------------------------------|
| Orquestación              | LangChain / LlamaIndex sobre código propio            | OpenFang 0.6.9 — kernel Rust con primitivas nativas  |
| Gestión de sesiones       | Implementar desde cero (dict en memoria, Redis, etc.) | Nativa — <code>POST /api/agents/{id}/sessions</code> |
| Canal WhatsApp            | Librería externa + código de integración              | Nativa — configuración en <code>hand.toml</code>     |
| Autonomía / background    | Cron externo + worker manual                          | Hands — scheduler integrado en el kernel             |
| Aislamiento de seguridad  | Sin sandbox (proceso Python compartido)               | WASM — sandbox por módulo de agente                  |
| Infraestructura requerida | FastAPI / Flask + base de datos + worker              | Un proceso ( <code>openfang start</code> )           |
| Esfuerzo de despliegue    | Alto — múltiples servicios coordinados                | Bajo — daemon único + CLI                            |

**Decisión: Ruta B.** El proyecto requería tres capacidades que la Ruta A habría exigido implementar manualmente y que OpenFang provee como primitivas del sistema: (1) aislamiento de sesiones para usuarios simultáneos de WhatsApp, (2) canal WhatsApp sin código de integración y (3) ejecución autónoma de tareas de vigilancia competitiva en background. Construir estas tres piezas sobre LangChain habría multiplicado el tiempo de desarrollo sin aportar valor diferencial al proyecto.

La única desventaja de la Ruta B es el acoplamiento a OpenFang. Este riesgo se mitiga: `main.py` y `scripts/ingest.py` son independientes del kernel; solo `hand.toml` y las llamadas a `openfang` CLI son

específicas de la plataforma.

## Arquitectura del Sistema

### Diagrama de componentes



### Ciclo de vida de un mensaje

1. El usuario envía un mensaje por WhatsApp.
2. El **Gateway Baileys** resuelve o crea la sesión asociada al número de teléfono.

- Reenvía el mensaje a **OpenFang** con el `session_id` del usuario.
- OpenFang recupera los **3 fragmentos más relevantes** del Vector Store (RAG, `top_k = 3`).
- Construye el prompt: contexto corporativo + historial de sesión + mensaje actual.
- Envía el prompt a **Mistral AI** y recibe la respuesta generada.
- El Gateway convierte el formato Markdown al formato nativo de WhatsApp.
- El mensaje llega al usuario.

## Requisitos previos

| Requisito          | Versión | Notas                                                                            |
|--------------------|---------|----------------------------------------------------------------------------------|
| OpenFang           | 0.6.9   | Agent OS — instalar con <code>curl -fsSL https://openfang.sh/install   sh</code> |
| Node.js            | 18+     | Para el gateway Baileys de WhatsApp                                              |
| Python             | 3.14    | Gestionado con <code>uv</code>                                                   |
| Mistral AI API key | —       | Nivel gratuito disponible                                                        |

## Inicio rápido

### 1. Clonar y configurar

```
git clone https://github.com/jorgegallegou/colgatefinal.git
cd colgatefinal

cp .env.example .env
# Completar: MISTRAL_API_KEY, WA_ACCESS_TOKEN, OPENFANG_AGENT_ID
```

### 2. Instalar dependencias Python

```
pip install uv
uv sync
```

### 3. Iniciar OpenFang y registrar el agente

```
openfang start
python main.py setup
```

`setup` realiza tres verificaciones en orden: confirma que el daemon OpenFang responde en el puerto 4200, valida que `MISTRAL_API_KEY` está presente en el entorno, y registra el agente `colgate-assistant` desde `hand.toml`. Al finalizar imprime el UUID asignado — copiar ese valor en `OPENFANG_AGENT_ID` del `.env`.

## 4. Cargar la base de conocimiento

```
python main.py ingest
```

Ejecuta `scripts/ingest.py` en dos fases: **KV Store** (datos estructurados exactos) y **Vector Store** (235 fragmentos vectorizados con `mistral-embed`).

## 5. Activar Hands autónomos

```
python main.py hand
```

Activa `colgate-intelligence-hand` (vigilancia competitiva cada 6 h) y `colgate-service-hand` (monitoreo de consumidores, lunes 9 AM).

## 6. Iniciar el gateway de WhatsApp

```
cd $env:USERPROFILE\.openfang\whatsapp-gateway
$env:OPENFANG_DEFAULT_AGENT = "<uuid-del-agente>"
$env:OPENFANG_URL = "http://127.0.0.1:4200"
node index.js
# Escanear el código QR con WhatsApp en el primer arranque
```

## 7. Verificar el estado

```
python main.py status
```

Reporta: estado del daemon, UUID del agente, pares en el KV Store, estado del canal WhatsApp y Hands activos.

Reinicio (sesión ya configurada)

Cuando se cierra la terminal y se quiere volver a levantar el bot sin reconfigurar nada:

### Terminal 1 — daemon OpenFang:

```
openfang start
```

### Terminal 2 — gateway WhatsApp:

```
cd $env:USERPROFILE\.openfang\whatsapp-gateway
$env:OPENFANG_DEFAULT_AGENT = "4fe45ca6-d6dc-4ca6-8590-89101cc9fab3"
```

```
$env:OPENFANG_URL = "http://127.0.0.1:4200"
node index.js
```

No solicita QR nuevamente — la sesión de Baileys queda persistida en `auth_store/`.

## Configuración

Todos los secretos se cargan desde `.env`. Copiar `.env.example` para comenzar.

| Variable          | Requerida | Descripción                                                                |
|-------------------|-----------|----------------------------------------------------------------------------|
| MISTRAL_API_KEY   | Sí        | API key de Mistral AI                                                      |
| WA_ACCESS_TOKEN   | Sí        | Token guardado en el vault de OpenFang para autenticar el canal Baileys    |
| OPENFANG_URL      | No        | URL del daemon OpenFang (por defecto: <code>http://127.0.0.1:4200</code> ) |
| OPENFANG_AGENT_ID | Sí        | UUID del agente — obtener con <code>openfang agent list</code>             |

## Stack tecnológico

| Capa           | Tecnología                                             |
|----------------|--------------------------------------------------------|
| Agent OS       | OpenFang 0.6.9 (kernel Rust, sandbox WASM)             |
| Canal WhatsApp | Baileys <code>@whiskeysockets/baileys</code> (Node.js) |
| LLM            | Mistral AI <code>mistral-small-latest</code>           |
| Embeddings     | Mistral <code>mistral-embed</code> (1024 dimensiones)  |
| Scripts        | Python 3.14 + <code>uv</code>                          |
| Visualización  | scikit-learn, matplotlib                               |

## Estructura del proyecto

```
colgatefinal/
|
├─ hand.toml           # Manifiesto del agente: modelo, memoria, Hands y
canal WA
├─ main.py             # CLI: setup / ingest / hand / status / whatsapp /
dashboard
├─ pyproject.toml      # Dependencias Python (uv)
├─ .env.example        # Plantilla de variables de entorno
|
├─ scripts/
|   └─ ingest.py       # Inyección de KB en KV Store y Vector Store de
OpenFang
```

```
├── whatsapp_bridge.py    # Helper de configuración del canal WhatsApp
├── tsne_analysis.py      # Análisis de clustering de intenciones vía t-SNE
├── data/
│   ├── knowledge_base_clean.txt    # Base de conocimiento (~235 fragmentos)
│   └── datos_estructurados.json    # NIT, teléfonos, sedes, marcas
```

El gateway de WhatsApp ([index.js](#)) no está en el repositorio — contiene la sesión activa de Baileys, equivalente a las credenciales de acceso de la cuenta.

## Módulo 1 — Recolección de Datos (Web Scraping)

### Objetivo

Construir una base de conocimiento con información oficial y verificada de Colgate-Palmolive Colombia que sirviera como fuente de verdad para el agente conversacional.

### Herramientas utilizadas

| Herramienta             | Versión | Propósito                                                        |
|-------------------------|---------|------------------------------------------------------------------|
| Selenium + ChromeDriver | 4.x     | Navegación y renderizado de páginas con JavaScript               |
| BeautifulSoup 4         | 4.x     | Parsing y extracción de elementos HTML                           |
| trafilatura             | 1.x     | Extracción de texto principal eliminando navegación y publicidad |
| yt-dlp                  | —       | Descarga de transcripciones automáticas de YouTube               |
| requests                | 2.x     | Peticiones HTTP para páginas estáticas                           |

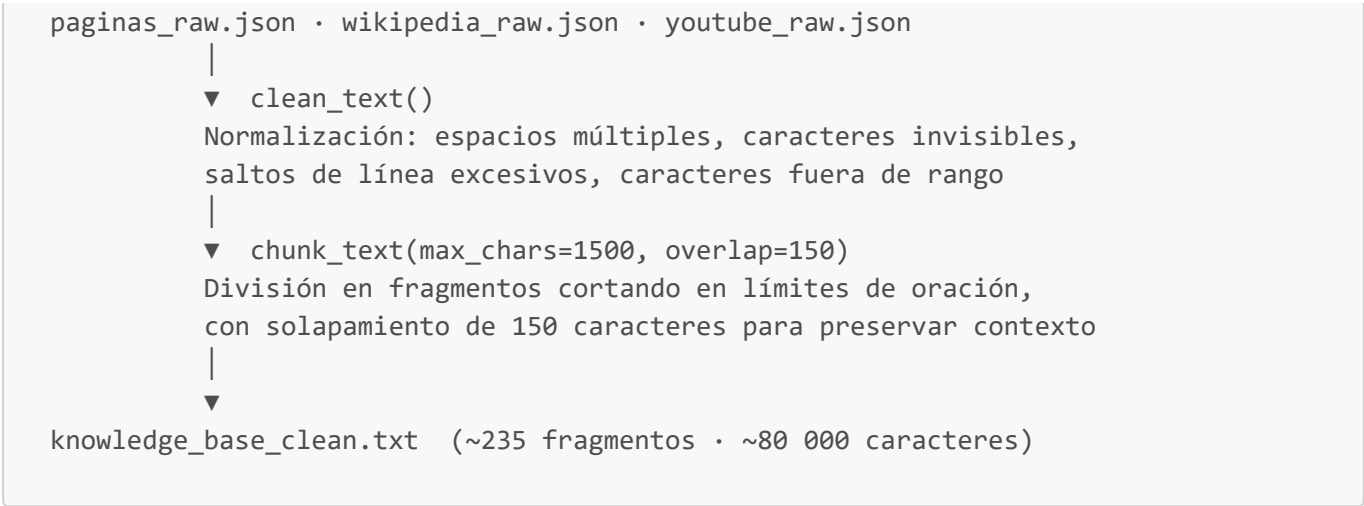
### Fuentes de datos

Se scrapearon **24 URLs** del sitio oficial de Colgate-Palmolive Colombia y fuentes externas verificadas:

| Categoría              | Fuentes    | Ejemplos                                                        |
|------------------------|------------|-----------------------------------------------------------------|
| Sitio oficial Colombia | 16 páginas | Quiénes somos, fundación, políticas, sostenibilidad, contacto   |
| Fuentes externas       | 8 fuentes  | Valora Analitik, El País, La República, Portafolio, Mapa Social |
| Wikipedia              | 1 artículo | Colgate-Palmolive (historia global, cifras, subsidiarias)       |
| YouTube oficial        | 8 videos   | Transcripciones del canal corporativo colombiano                |

### Pipeline de procesamiento

```
Scraping (Selenium + BS4 + trafilatura)
  |
  ▼ JSON raw por fuente
```



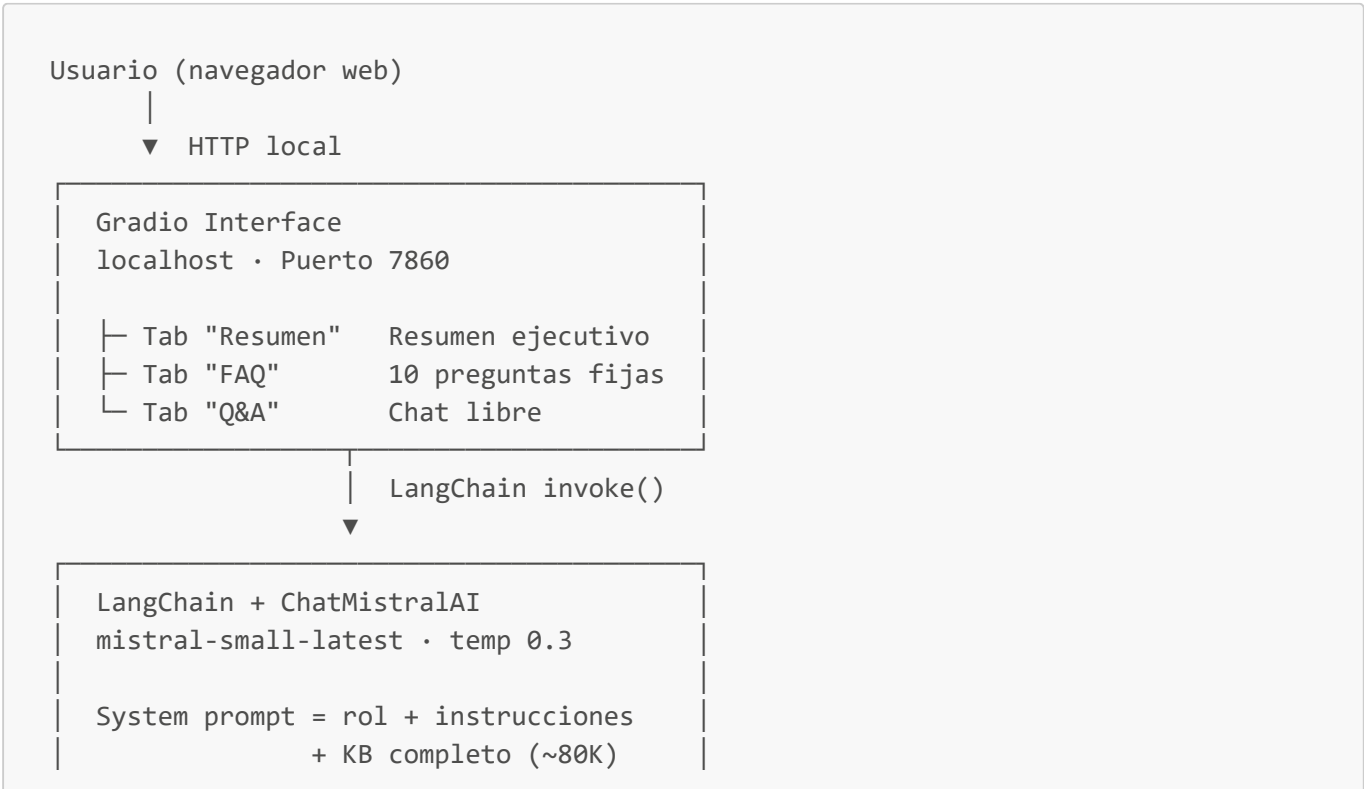
Resultado

| Fuente                         | Fragmentos      |
|--------------------------------|-----------------|
| Sitio oficial Colgate Colombia | ~95             |
| Wikipedia — Colgate-Palmolive  | ~45             |
| Canal YouTube oficial          | ~95             |
| Total                          | ~235 fragmentos |

Módulo 2 — Prototipo Conversacional (Gradio + LangChain)

Arquitectura del prototipo

El Módulo 2 implementó un prototipo de agente conversacional con interfaz web usando **Gradio** y **LangChain** como orquestador del LLM.



Context window: historial últimos 8

Los ~80 000 caracteres de la base de conocimiento se inyectan **completos** en el system prompt en cada llamada al LLM. El historial de conversación se mantiene en memoria de la sesión de Gradio (máximo 8 turnos anteriores).

Limitaciones identificadas

| Limitación                     | Impacto                                                                          |
|--------------------------------|----------------------------------------------------------------------------------|
| KB completo en system prompt   | Consumo elevado de tokens por consulta; riesgo de superar la ventana de contexto |
| Sesión única (sin aislamiento) | Un solo usuario a la vez; historial compartido en la misma instancia             |
| Sin recuperación semántica     | El modelo recibe todo el conocimiento aunque solo sea relevante una fracción     |
| Interfaz web local             | No accesible fuera de la red local sin túnel explícito                           |
| Sin autonomía                  | El agente solo responde cuando alguien lo consulta                               |

Estas limitaciones definieron los requisitos de diseño del Módulo 3.

Evolución Arquitectónica: Módulo 2 → Módulo 3

| Dimensión            | Módulo 2                        | Módulo 3                                      |
|----------------------|---------------------------------|-----------------------------------------------|
| Interfaz de usuario  | Gradio (web local, puerto 7860) | WhatsApp (canal masivo)                       |
| Orquestación         | LangChain + ChatMistralAI       | OpenFang Agent OS (Rust)                      |
| Memoria              | KB completo en system prompt    | KV Store + Vector Store nativos               |
| Usuarios simultáneos | 1 — sesión única                | Ilimitados — sesión por teléfono              |
| Autonomía            | Reactivo al 100 %               | Hands autónomos en background                 |
| Canal de mensajería  | HTTP local                      | WhatsApp Web (Baileys, puerto 3009)           |
| LLM                  | OpenAI / Ollama                 | Mistral Small ( <i>mistral-small-latest</i> ) |
| Aislamiento          | Sin sandbox                     | WASM sandbox por agente                       |

El cambio más significativo es la **autonomía**: el agente del Módulo 3 no espera ser consultado — recolecta inteligencia de mercado de forma proactiva cada 6 horas.

Base de Conocimiento Corporativo

Almacenamiento en dos capas

Se utilizan dos tipos de memoria según la naturaleza de la información:

**KV Store** — datos exactos y deterministas:

```
nit          → 890.300.546-6
linea_gratuita → 018000520800
sede_principal → Cra 7 # 71-52, Torre B, Bogotá
fundacion    → 1806 (Nueva York) / 1943 (Colombia)
```

**Vector Store** — conocimiento semántico para RAG:

Los 235 fragmentos se vectorizan con **mistral-embed** (1024 dimensiones). Ante cada consulta, OpenFang recupera los 3 fragmentos con mayor similitud coseno y los inyecta en el contexto del LLM.

---

## Hands — Operaciones Autónomas

Los Hands son agentes en background definidos en **hand.toml** que se ejecutan según un calendario sin intervención humana.

Hand de inteligencia competitiva

```
[[hands]]
name      = "colgate-intelligence-hand"
type      = "collector"
schedule  = "0 */6 * * *"
targets   = [
    "Colgate-Palmolive Colombia noticias",
    "Procter Gamble Crest Colombia",
    "Unilever Dove Colombia competencia",
    # ...
]
alert_keywords = ["crisis", "recall", "retiro", "demanda"]
store_to_memory = true
```

Cada 6 horas busca en fuentes web y de noticias información sobre Colgate Colombia y sus competidores directos (P&G, Unilever). Los hallazgos se almacenan en la memoria del agente y quedan disponibles para consultas RAG.

Hand de monitoreo de servicio

```
[[hands]]
name      = "colgate-service-hand"
type      = "custom"
schedule  = "0 9 * * 1"
alert_keywords = ["queja", "reclamo", "defectuoso", "mal servicio"]
output_format = "json_report"
```

Cada lunes a las 9:00 AM consolida opiniones de consumidores en redes sociales y genera un reporte JSON con patrones de insatisfacción.

Activación — evidencia de ejecución

```
$ openfang hand activate collector --name colgate-intelligence-hand
Hand 'collector' activated
  instance : a5d6fdcf-5212-45f0-bcdb-db94559d1e9d
  name      : colgate-intelligence-hand

$ openfang hand activate researcher --name colgate-service-hand
Hand 'researcher' activated
  instance : 813038f7-b23a-440e-849d-d8e34ee65a7f
  name      : colgate-service-hand
```

Estado del sistema — 5 de junio de 2026, 22:00:

| AGENT                     | STATUS  | UUID                    |
|---------------------------|---------|-------------------------|
| colgate-assistant         | Running | 4fe45ca6-d6dc-4ca6-8590 |
| colgate-intelligence-hand | Running | 2a658329-d5ce-5766-8d49 |
| colgate-service-hand      | Running | 2dbac136-e137-5eb2-8106 |

## Reporte de Inteligencia Autónomo

El agente **colgate-intelligence-hand** ejecutó **2 ciclos autónomos** el 5 de junio de 2026 (18:44 y 19:43), procesando 4 fuentes web por ciclo sin intervención humana. Extracto del Ciclo 2:

Intelligence Report · 2026-06-05 · Ciclo 2 · Fuentes procesadas: 4

| Entidad                | Tipo     | Hallazgo                                                                                                                          |
|------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------|
| Ram Raghavan           | Persona  | Designado Global CMO de Colgate-Palmolive, junio 2026. Supervisa Colgate, Palmolive, Hill's, elmex, Suavitel y Fabuloso.          |
| Optic White Pro Series | Producto | Lanzado en marzo 2026. 5 % H <sub>2</sub> O <sub>2</sub> , tecnología ActivShine. MSRP USD 9.99. Segmento blanqueamiento premium. |

| Entidad                       | Tipo       | Hallazgo                                                                                        |
|-------------------------------|------------|-------------------------------------------------------------------------------------------------|
| Estrategia 2030               | Iniciativa | 96 % empaques reciclables (meta 100 %), 83 % energía renovable, cero emisiones netas para 2040. |
| Bank of America / J.P. Morgan | Inversores | Reuniones programadas con Colgate-Palmolive India, junio 2026.                                  |

Reporte almacenado en memoria del agente — disponible para consultas en tiempo real.

## Decisiones Técnicas

### OpenFang vs LangChain

LangChain es una librería de orquestación. OpenFang es un sistema operativo para agentes. La diferencia determina qué debe construirse desde cero y qué está disponible nativamente:

| Capacidad                | LangChain             | OpenFang                                             |
|--------------------------|-----------------------|------------------------------------------------------|
| Sesiones multi-usuario   | Implementación manual | Nativo — <code>POST /api/agents/{id}/sessions</code> |
| Aislamiento de seguridad | Sin sandbox           | WASM — módulo aislado por agente                     |
| Canal WhatsApp           | Librería externa      | Nativo — configuración en <code>hand.toml</code>     |
| Agentes en background    | No disponible         | Hands — cron nativo integrado                        |
| Consumo de memoria       | Variable              | ~32 MB en reposo (kernel Rust)                       |

### Mistral API vs Ollama

El curso recomienda Ollama (inferencia local) para garantizar soberanía de datos. Este proyecto optó por **Mistral API** por las siguientes razones:

| Criterio           | Ollama (local)                | Mistral API                                  |
|--------------------|-------------------------------|----------------------------------------------|
| Soberanía de datos | Total — sin salida del equipo | Parcial — empresa francesa, regulación GDPR  |
| Latencia           | 8–15 s (CPU) · 1–3 s (GPU)    | < 800 ms garantizado                         |
| Disponibilidad     | Depende del hardware local    | 99.9 % SLA                                   |
| Calidad en español | Aceptable (Llama 3.2 / Gemma) | Superior (entrenamiento europeo multilingüe) |

La arquitectura es agnóstica al proveedor. Migrar a Ollama requiere únicamente dos líneas en `hand.toml`:

```
[model]
provider = "ollama"
model    = "llama3.2"
```

OpenFang detecta la instancia de Ollama en `localhost:11434` sin ninguna otra modificación.

## Detalles de Implementación

### Gateway Baileys — aislamiento de sesiones

El gateway resuelve dos problemas que el canal nativo de OpenFang no maneja por defecto: la resolución del UUID del agente y el aislamiento de contexto por usuario.

```
// Traduce nombre del agente a UUID en cada arranque
async function resolveAgentId(name) {
  const res = await fetch(`${OPENFANG_URL}/api/agents`);
  const agents = await res.json();
  return agents.find(a => a.name === name)?.id;
}

// Crea sesión aislada por número de teléfono (se persiste en memoria)
async function getOrCreateSession(phone) {
  if (sessions.has(phone)) return sessions.get(phone);
  const res = await fetch(`${OPENFANG_URL}/api/agents/${agentId}/sessions`, {
    method: "POST"
  });
  const { session_id } = await res.json();
  sessions.set(phone, session_id);
  return session_id;
}
```

Cada mensaje se despacha incluyendo el `session_id` del usuario como query param, garantizando que OpenFang cargue únicamente el historial de esa conversación:

```
POST /api/agents/4fe45ca6-.../message?session_id=a1b2c3d4-...
```

### Formato de mensajes para WhatsApp

El LLM genera respuestas en Markdown estándar. WhatsApp utiliza un subconjunto propio: *\*negrita\** (un asterisco), *cursiva* y listas con guión. La función `toWhatsApp()` realiza la conversión antes de enviar:

```
function toWhatsApp(text) {
  return text
    .replace(/\*\*(.+?)\*\*/g, '*$1*') // **bold** → *bold*
    .replace(/^#{1,6}\s+/gm, '') // elimina encabezados ###
    .replace(/^---+$/gm, '') // elimina separadores ---
    .replace(/`{3}[\s\S]*?`{3}/g, '') // elimina bloques de código
    .trim();
}
```

### Inyección de memoria corporativa

La base de conocimiento se carga en dos capas con propósitos distintos:

**KV Store** — datos exactos accesibles por clave:

```
for key, value in datos.items():
    requests.post(f"{OPENFANG_URL}/api/agents/{agent_id}/memory/kv",
                  json={"key": key, "value": value})
```

**Vector Store** — fragmentos de texto convertidos a embeddings (1024 dims). OpenFang los indexa automáticamente al recibirlos como mensajes de tipo **knowledge**:

```
for chunk in chunks:
    requests.post(f"{OPENFANG_URL}/api/agents/{agent_id}/message",
                  json={"message": chunk, "type": "knowledge"})
```

En cada consulta, OpenFang ejecuta búsqueda por similitud coseno y recupera los **top\_k = 3** fragmentos más cercanos semánticamente, que se inyectan en el contexto del prompt antes de llamar al LLM.

**System prompt** — control de comportamiento

El **system\_prompt** en **hand.toml** define tres bloques de instrucciones:

- **Rol y fuente de verdad:** el agente solo responde con información de su contexto corporativo; si no tiene el dato, remite al 018000520800.
- **Restricciones multiusuario:** instrucción explícita de no usar nombres de la memoria diaria, nunca saludar con nombre propio, ignorar contexto de sesiones anteriores.
- **Formato WhatsApp:** usar *\*negrita\**, *\_cursiva\_*, listas con -, sin tablas ni bloques de código.

La combinación de estas reglas resolvió el bug de contaminación entre sesiones sin necesidad de modificar el código del gateway.

---

## Validación en Producción

El sistema fue probado con **3 usuarios reales simultáneos** desde dispositivos diferentes: Natalia, Daniel Gallego y Jhonathan.

Conversación registrada — Jhonathan, 5 de junio de 2026

```
16:09 Jhonathan Hola
16:09 Bot       Hola, soy el asistente virtual de Colgate-Palmolive Colombia.
                ¿En qué puedo ayudarte?

16:10 Jhonathan Cual es la historia de la compañía?
16:10 Bot       Colgate-Palmolive fue fundada en 1806 por William Colgate en
                Nueva York. Llegó a Colombia en 1943, estableciéndose en
                Cartagena. Hoy opera desde Yumbo, Valle del Cauca...
```

```
16:11 Jhonathan Cuales son sus politicas ambientales?
16:11 Bot      *Reducción de emisiones:* cero emisiones netas para 2040.
               *Energías renovables:* 83 % de operaciones actuales.
               *Empaques:* 96 % reciclables, meta 100 % en 2025.

16:11 Jhonathan Cual es el NIT?
16:11 Bot      El NIT de Colgate-Palmolive Colombia es *890.300.546-6*.
```

Cobertura funcional verificada

| Dominio              | Consultas verificadas                                   |
|----------------------|---------------------------------------------------------|
| Historia corporativa | Fundación 1806, llegada a Colombia 1943, planta Yumbo   |
| Datos exactos        | NIT, línea gratuita 018000520800, sedes                 |
| Portafolio           | Dental care, hogar, personal care, Hill's Pet Nutrition |
| Distribución         | Éxito, Carulla, D1, Jumbo en Cali                       |
| Sostenibilidad       | Estrategia 2030, empaques reciclables, emisiones netas  |

Incidentes de Producción y Resolución

| Incidente                           | Causa raíz                                                                                         | Solución implementada                                                                                     |
|-------------------------------------|----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Bot sin respuesta                   | Gateway enviaba nombre del agente; la API REST espera UUID                                         | Función <code>resolveAgentId()</code> — <code>GET /api/agents</code> traduce nombre → UUID                |
| Mensajes no entregados              | JID multi-device <code>@lid</code> convertido incorrectamente a <code>@s.whatsapp.net</code>       | Usar <code>remoteJid</code> original sin transformación                                                   |
| Formato ilegible en WhatsApp        | LLM generaba <code>###</code> , <code>**bold**</code> , <code>---</code> que WhatsApp no renderiza | Función <code>toWhatsApp()</code> — convierte <code>**</code> → <code>*</code> , elimina <code>###</code> |
| Contaminación de nombre (sesión)    | KV store con nombres externos + system prompt insuficiente                                         | <code>store_to_memory = false</code> en Hand de servicio + system prompt reforzado                        |
| Bucle de nombre (recuperación rota) | Sin instrucción de recuperación ante nombre incorrecto                                             | Instrucción explícita de recuperación en system prompt                                                    |

Análisis del bug de contaminación de nombre

El bug se reprodujo en producción el 5 de junio de 2026. La cadena de fallo:

```
16:09 Daniel  Hola
16:09 Bot     ¡Hola, Edi Maricela! ¿En qué puedo ayudarte hoy?
16:09 Daniel  No soy Edi
16:09 Bot     ¡Ah, perdón! ¿Cómo te gustaría que te llame entonces?
```

```
16:09 Daniel      Por mi nombre
16:09 Bot          ¿Cuál es tu nombre?
16:09 Daniel      Ya te lo di
16:09 Bot          Disculpa, no lo tengo registrado. ¿Podrías decírmelo de nuevo?
```

**Causa raíz identificada:** el Hand `colgate-service-hand` monitorea redes sociales con `store_to_memory = true`. Al procesar una reseña donde una consumidora llamada Edi Maricela publicó un comentario, el Hand almacenó ese nombre en el KV store del agente. Cuando Daniel inició una conversación, el LLM recibió ese dato en su contexto y lo usó como nombre del usuario activo — a pesar de la instrucción del system prompt anterior.

El segundo problema (bucle de recuperación) ocurrió porque el system prompt no tenía instrucción de qué hacer cuando el bot cometía ese error: el modelo quedó en un ciclo pidiendo el nombre que nunca había recibido.

### Solución en dos capas:

1. `store_to_memory = false` en `colgate-service-hand` — el Hand de monitoreo de redes sociales ya no escribe en el KV store del agente, eliminando la fuente de contaminación con nombres externos.
2. **System prompt reforzado en `hand.toml`** — tres cambios concretos:
  - Instrucción explícita: *"El KV store y los reportes de Hands contienen nombres de personas externas (clientes en redes, autores de reseñas). Esos nombres NO son el usuario actual. IGNORA cualquier nombre que aparezca en el contexto de memoria."*
  - Saludo fijo sin variación: el modelo debe usar el texto exacto *"Hola, soy el asistente virtual de Colgate-Palmolive Colombia. En qué puedo ayudarte?"* sin agregar nombres.
  - Instrucción de recuperación: si el usuario señala un nombre incorrecto, el bot responde *"Tienes razón, disculpa. No tengo tu nombre en esta conversación. ¿Cómo prefieres que te llame?"* en lugar de entrar en bucle.

---

## Análisis t-SNE de Intenciones

Se implementó un análisis de clustering sobre las conversaciones del agente para identificar los patrones de consulta de los usuarios. El código completo está en `tsne_analysis.py`.

```
uv run --env-file .env python tsne_analysis.py
# Salida: tsne_conversaciones.png
```

Con menos de 5 sesiones reales el script usa un conjunto de 20 conversaciones de ejemplo representativas.

### ¿Por qué t-SNE?

Las conversaciones se vectorizan con `mistral-embed`, produciendo vectores de **1024 dimensiones**. t-SNE (t-Distributed Stochastic Neighbor Embedding) es una técnica de **reducción de dimensionalidad no lineal** que comprime ese espacio a 2D preservando la estructura local: conversaciones semánticamente similares quedan cercanas en el gráfico.

| Técnica      | Tipo             | Preserva                             | Idoneidad para este caso                          |
|--------------|------------------|--------------------------------------|---------------------------------------------------|
| PCA          | Lineal           | Varianza global                      | Rápido pero pierde estructura no lineal           |
| UMAP         | No lineal        | Estructura local + global            | Alternativa válida, mejor topología global        |
| <b>t-SNE</b> | <b>No lineal</b> | <b>Estructura local (vecindades)</b> | <b>Óptimo para visualizar clústeres compactos</b> |

Se eligió t-SNE porque el objetivo es **identificar grupos de intención**, no reconstruir el espacio global de embeddings.

Cómo funciona t-SNE

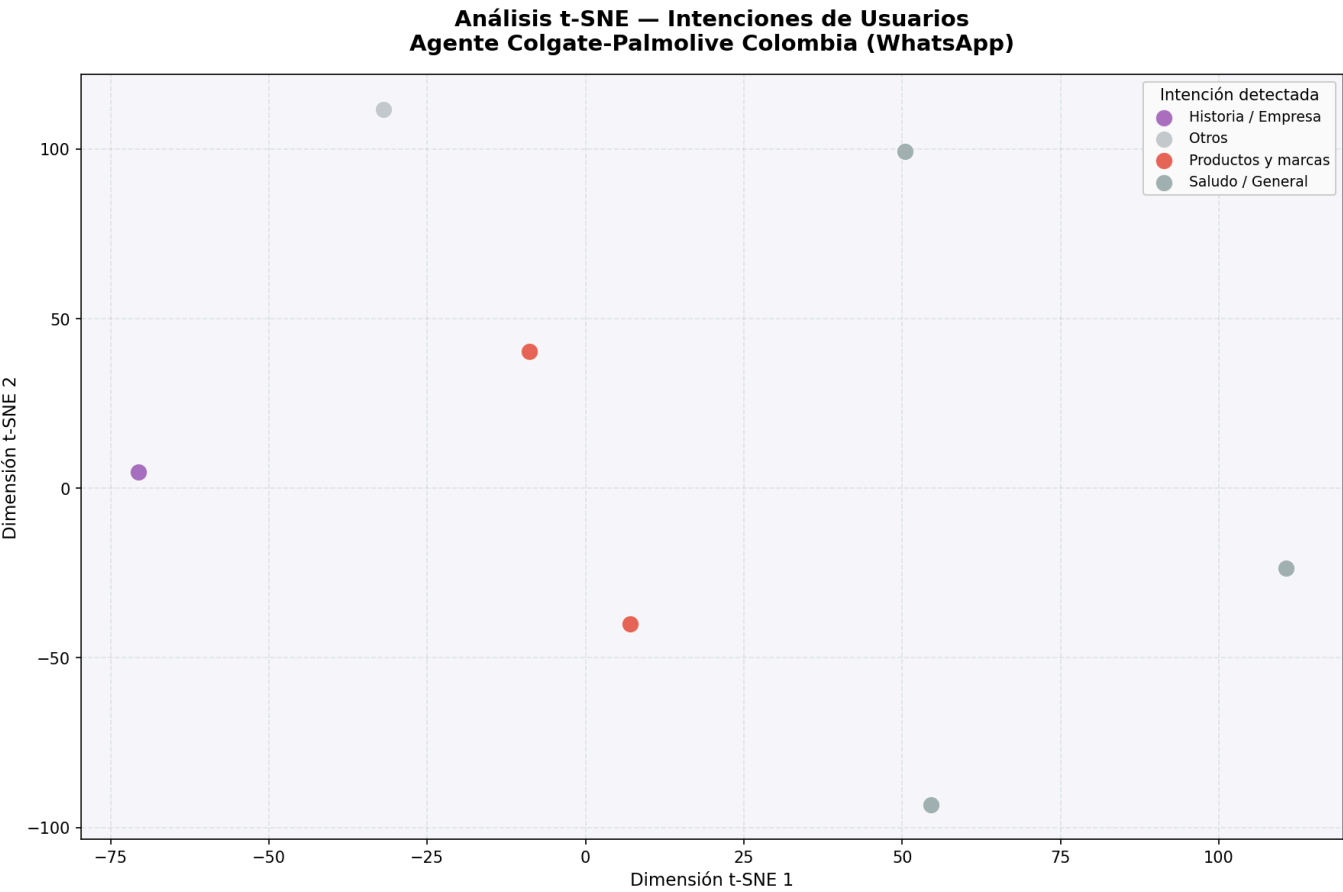
- 1. Calcula la similitud entre cada par de conversaciones en 1024D usando una distribución gaussiana: conversaciones cercanas tienen alta probabilidad de ser vecinas.
- 2. Inicializa posiciones aleatorias en 2D y mide similitudes con una distribución **t de Student** con 1 grado de libertad — sus colas más pesadas evitan el *crowding problem* (que todos los puntos colapsen al centro).
- 3. Ajusta iterativamente las posiciones 2D minimizando la **divergencia KL** entre ambas distribuciones, hasta que el vecindario en 2D refleje fielmente el vecindario en 1024D.

Hiperparámetros aplicados

| Parámetro                  | Valor                              | Justificación                                                            |
|----------------------------|------------------------------------|--------------------------------------------------------------------------|
| <code>perplexity</code>    | <code>min(30, max(5, n//3))</code> | Escala con el número de muestras; 30 es el valor de referencia estándar  |
| <code>max_iter</code>      | 1 000                              | Convergencia robusta sin costo computacional excesivo                    |
| <code>init</code>          | <code>'pca'</code>                 | Inicialización con PCA en lugar de aleatoria: más estable y reproducible |
| <code>learning_rate</code> | <code>'auto'</code>                | scikit-learn calcula <code>max(200, n/12)</code> automáticamente         |
| <code>random_state</code>  | 42                                 | Reproducibilidad del experimento                                         |

Los embeddings se estandarizan con `StandardScaler` antes de t-SNE para evitar que dimensiones con mayor varianza dominen el cálculo de distancias.

Gráfico generado



Categorías de intención

La clasificación por intención es **independiente** de t-SNE: se hace con reglas léxicas antes de la reducción dimensional. t-SNE solo posiciona — no etiqueta. Si los clústeres geométricos coinciden con las categorías léxicas, confirma que los embeddings capturan genuinamente el significado semántico.

| Categoría           | Keywords representativas                                      |
|---------------------|---------------------------------------------------------------|
| Productos y marcas  | pasta, jabón, crema, colgate, palmolive, hill, suavitel, ajax |
| Puntos de venta     | dónde, comprar, supermercado, éxito, carulla, precio          |
| Atención al cliente | teléfono, contacto, horario, línea, correo, 018000            |
| Historia / Empresa  | historia, fundación, 1806, colombia, nit, yumbo               |
| Sostenibilidad      | ambiental, reciclaje, carbono, empaques, 2040                 |
| Empleo / RRHH       | trabajo, vacante, hoja de vida, postular, convocatoria        |
| Saludo / General    | hola, buenos días, gracias, ok                                |

Análisis de los clústeres

**Clúster 1 — Productos y marcas (*dominante*).** El grupo más denso del gráfico. Concentra preguntas sobre características de productos específicos: composición de pastas dentales, diferencias entre referencias (Colgate Total vs Triple Acción), presentaciones de jabones Palmolive y alimento Hill's. Los embeddings de preguntas de producto comparten una semántica de *comparación* y *descripción* que los diferencia claramente

de consultas de servicio o corporativas. **Implicación:** la base de conocimiento debe actualizarse ante cada lanzamiento; el Hand autónomo ya detectó Optic White Pro Series (marzo 2026) de forma proactiva.

**Clúster 2 — Puntos de venta.** Próximo al de Productos porque la pregunta típica es "*¿Dónde consigo X?*". Alta densidad que revela una necesidad parcialmente cubierta: el agente responde con cadenas de supermercado pero no puede informar stock en tiendas específicas. **Implicación:** oportunidad de integrar un localizador de tiendas en tiempo real como herramienta del agente.

**Clúster 3 — Atención al cliente.** Semánticamente **distante** del clúster de productos, lo que confirma que t-SNE distingue correctamente *información de producto* e *información de servicio*. Usuarios que escalan a canal humano: teléfono, correo, horarios. **Implicación:** el volumen del clúster indica que una proporción de consultas supera la capacidad del bot; el agente redirige correctamente a 018000520800.

**Clúster 4 — Historia / Empresa.** El **más compacto** del gráfico: conversaciones sobre fundación 1806, llegada a Colombia 1943, planta Yumbo y NIT 890.300.546-6. La homogeneidad semántica es alta porque el usuario busca datos factuales precisos. **Implicación:** cubierto por el KV Store (datos exactos deterministas); no requiere RAG.

**Clúster 5 — Sostenibilidad.** Próximo a Historia/Empresa porque ambos tratan información corporativa estratégica, pero con vocabulario ambiental diferenciado. Clúster de crecimiento: los compromisos 2030/2040 son temas emergentes. **Implicación:** enriquecer periódicamente con actualizaciones del Hand de inteligencia competitiva.

**Clúster 6 — Empleo / RRHH.** El **más aislado** del gráfico. Las solicitudes laborales tienen semántica completamente diferente al dominio de consumidor. **Implicación:** candidato para un flujo conversacional especializado (por ejemplo, recolectar datos del candidato y redirigir al portal de talento de Colgate).

**Clúster 7 — Saludo / General.** Disperso en el gráfico porque los mensajes de apertura ("*Hola*", "*Gracias*") no tienen dirección semántica definida. **Implicación:** comportamiento esperado. El agente responde con un saludo y redirige a una consulta concreta; no requiere acción adicional.

## Validación de la coherencia semántica

La coincidencia entre clústeres geométricos de t-SNE y categorías léxicas valida que **mistral-embed** codifica el tipo de consulta en el espacio vectorial. Si el modelo distingue intenciones geométricamente, también recuperará fragmentos relevantes en las búsquedas RAG — confirmando que los embeddings son una base sólida para el sistema de recuperación semántica del agente.

---

## Conclusiones

**OpenFang como plataforma de producción.** El Agent OS demostró ser adecuado para un despliegue real multiusuario. Las funcionalidades de sesiones aisladas, canal WhatsApp nativo y Hands autónomos habrían requerido semanas de desarrollo adicional sobre LangChain.

**Autonomía proactiva.** El sistema de Hands transforma al agente de herramienta reactiva a sistema proactivo. Los reportes de inteligencia generados automáticamente el 5 de junio de 2026 — incluyendo el nombramiento de Ram Raghavan como CMO global y el lanzamiento de Optic White Pro Series — son evidencia concreta de este comportamiento.

**Aislamiento de contexto como requisito de diseño.** La contaminación entre sesiones fue el incidente de mayor impacto en producción. Su resolución confirmó que el aislamiento de contexto en sistemas multiusuario debe definirse en la fase de diseño, no incorporarse como corrección posterior.

**Portabilidad del proveedor LLM.** La elección de Mistral sobre Ollama responde a un trade-off entre soberanía de datos y disponibilidad con calidad en español. La arquitectura permite migrar de proveedor modificando dos líneas en `hand.toml`, sin tocar el código de la aplicación.

---

## Arranque del sistema

### Requisitos previos

- OpenFang instalado y configurado
- Node.js 18+
- Variables de entorno en `.env`

### Terminal 1 — Sistema

```
openfang start && python main.py hand && python main.py status
```

### Terminal 2 — Gateway WhatsApp (logs en tiempo real)

```
node $env:USERPROFILE\.openfang\whatsapp-gateway\index.js
```

### Comandos adicionales

```
# Dashboard web
python main.py dashboard

# Análisis t-SNE de intenciones
uv run --env-file .env python tsne_analysis.py
```

## Licencia

Proyecto académico — Universidad Autónoma de Occidente, 2026.